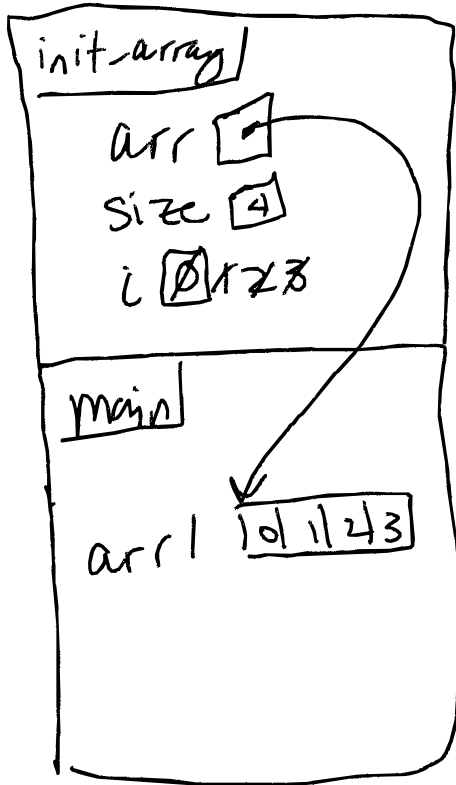


Passing arrays to functions



2D Arrays

2 approaches

- one call to malloc
 - efficient
 - harder to reason about
- one call to malloc for setting up the ptrs to the columns, and then one for each column itself
 - less efficient
 - easier to reason about
 - easier to index

Array of Arrays

0 1 2
0 $\begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$
1 $\begin{bmatrix} 4 & 5 & 6 \end{bmatrix}$

2x3

MxN

int arr[2][3] = {

{1, 2, 3},

{4, 5, 6}

};

nums[0][0] = 1

nums[1][2] = 6

Dynamic: first malloc(sizeof(int*)xM)

int** arr



Then: for (i = 0, i < M; i++)

arr[i] = malloc(sizeof(int)
x N)

Flat Array

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

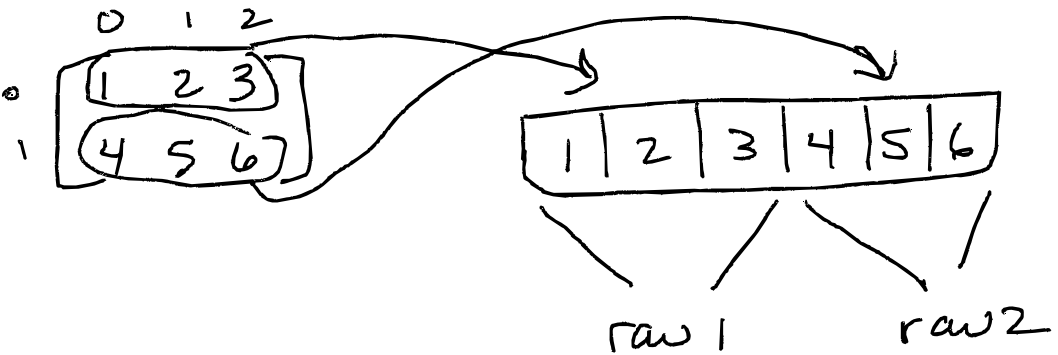
$M \times N$

2×3

malloc all at once :

`malloc(sizeof(int) * (2*3))`

`int* arr` → creates an array of 6 elements



but! we can't index it like

`arr[0][0]`

of
cols
↓

location = row $\times$ N + col

$$\text{arr}[0][0] = 0 \times 3 + 0 = 0$$

$$\text{arr}[1][2] = 1 \times 3 + 2 = 5$$

Different Types

$\text{int}^{**} \text{arr}$ vs $\text{int}^* \text{arr}$

Flat 2D Array:

Computing indices

$$\text{location} = \text{row} \times \# \text{col} + \text{col}$$

[4, 4] element (2, 3)

$$2 \times 4 + 3 = 11$$

[3, 10] element (2, 3)

$$2 \times 10 + 3 = 23$$

[10, 3] element (4, 0)

$$4 \times 3 + 0 = 12$$

ask_matrix_flat.c

- creates
- frees

3x4 matrix

```
int* flat_matrix;
```

```
flat_matrix =
```

```
    malloc(sizeof(int*) * 12);
```

```
free(flat_matrix);
```

ask-matrix-aa.c

array of arrays

random # program:

forward
declaration

Review

execution stack: stack
data structure that stores
info / data for active
functions

pointer: obj that stores a
memory address

Static vs dyn mem

allocation:

Static: we declare the array and give its size

immediately: `int arr [3] [4];`
↳ lives on stack

dynamic: we declare the array and then use malloc to allocate the memory
↳ lives on heap

pointer operators

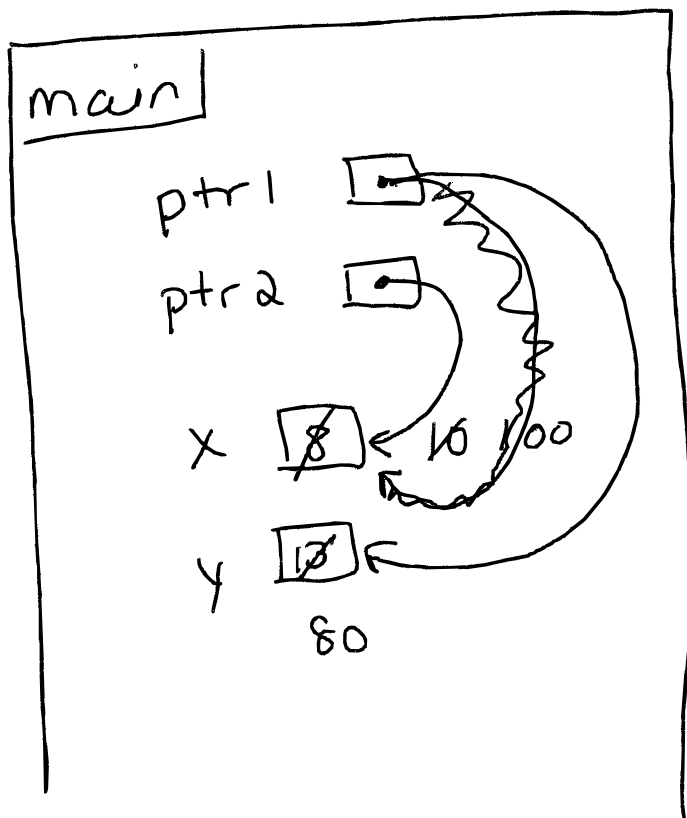
char c1 = 'a';

char* ptr; ← defining a ptr

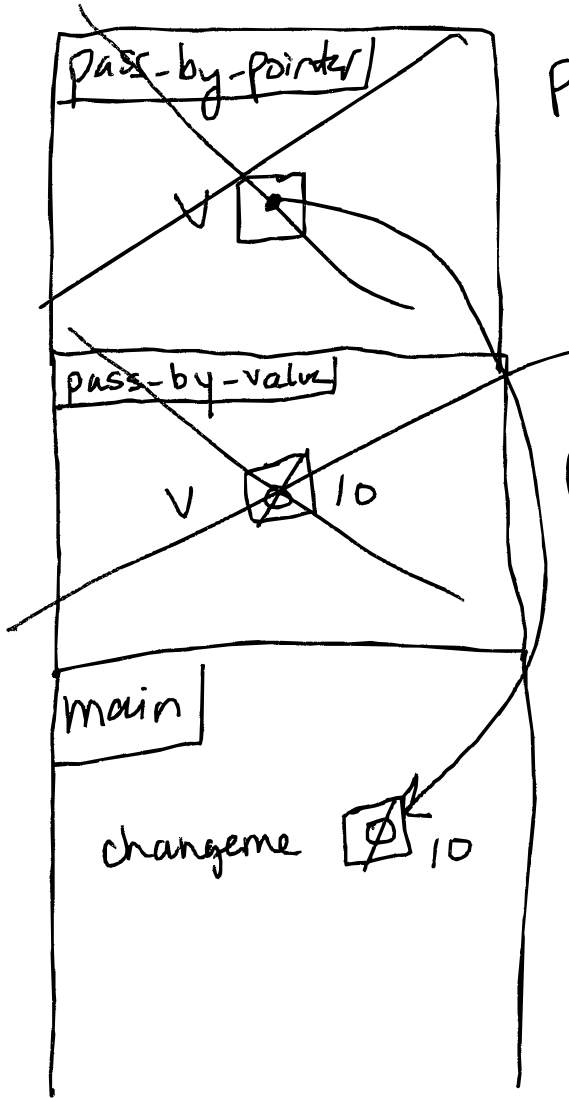
ptr = &c1; ← address operator

char c2 = *ptr; ← dereferencing
the ptr to get the
value

Exercise : draw a stack diagram
for this function



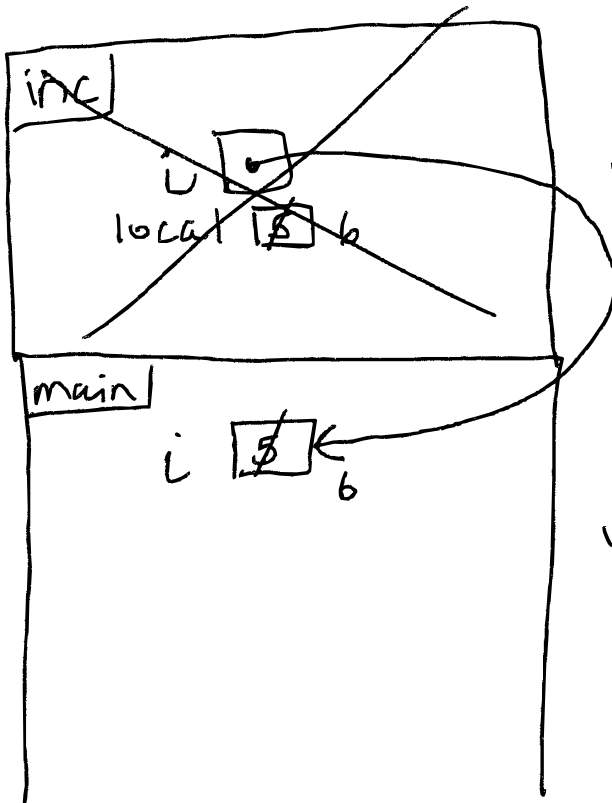
Pass by pointer: draw stack



prints : "the val
is 10"

prints : "the val
is 0"

Example: pass by pointer



prints:
"After: 6"

prints:
"Before: 5"

Struct pass by value

change Gpa

S:
gpa 3.5 3.7
newGpa: 3.7

prints
after
3.5

changeName

S: K W A M E I O
name R | V | T | H | I | O

prints
after
"Ruth"

newName R

main

student1:
name R | V | T | H | I | O
gpa 3.5

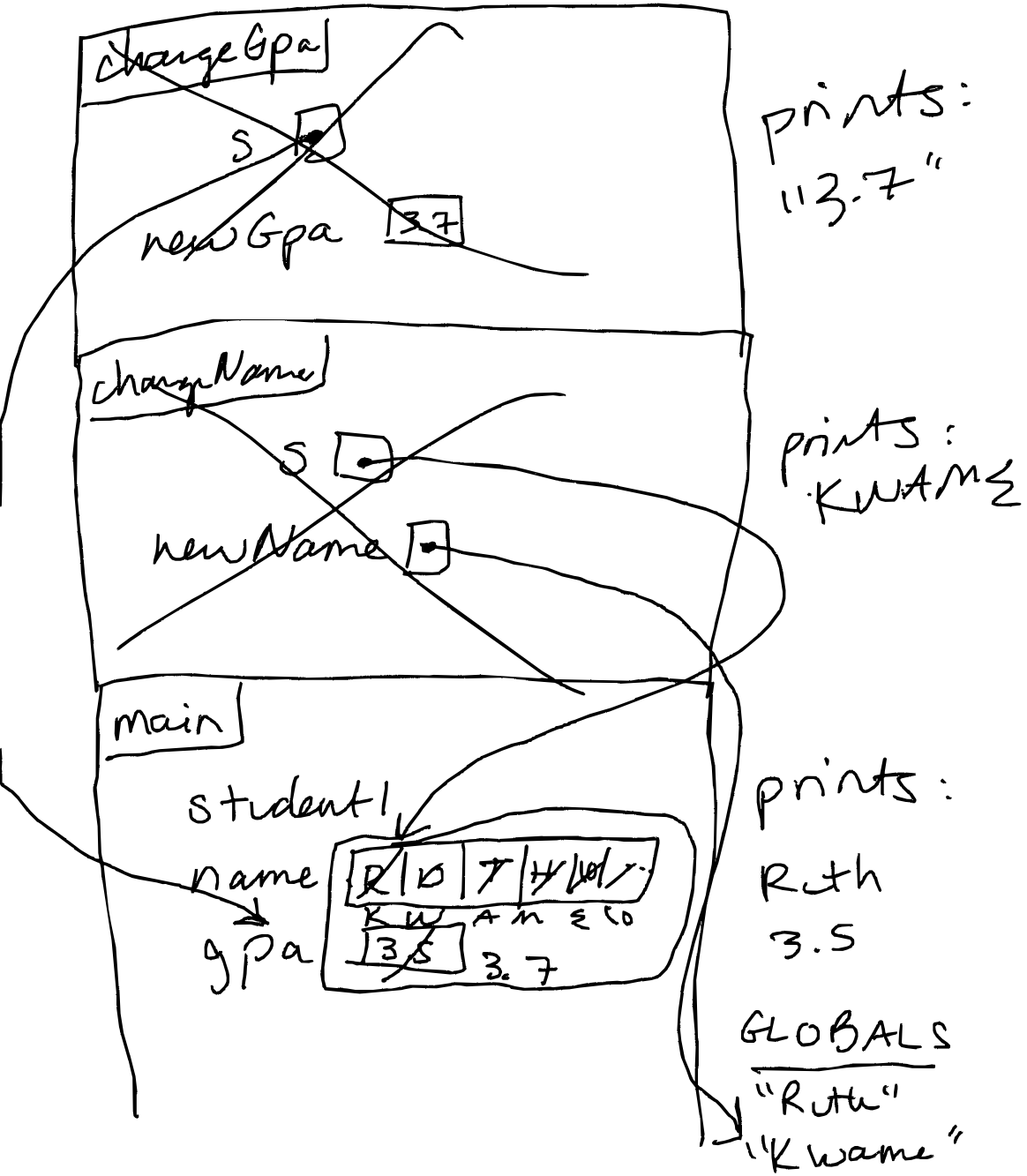
prints:

OG
"Ruth"

GLOBALS

"Ruth"
"Kwame"

Struct pass by pointer



Compiles? #1 NO

- struct passed as value
 - $\rightarrow$ is a pointer operation, should be dot
-

Compiles? #2 YES

- struct passed by ptr
- using ptr operation to update field in struct

Compiles? #3

(3 change names)

1) yes ✓

- struct passed by ptr
- → ptr op to update field

2) NO X

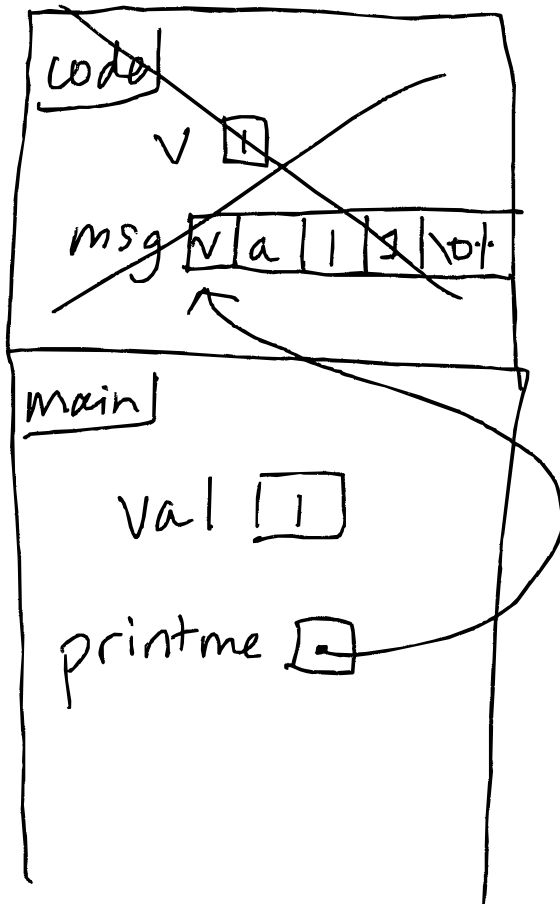
- struct passed by ptr
- can't use dot, need → operator

3) NO X

$s \rightarrow gpa = \&value$
type: float type: float* (ptr)

Return value gotchas #1

not safe x



read-only / static

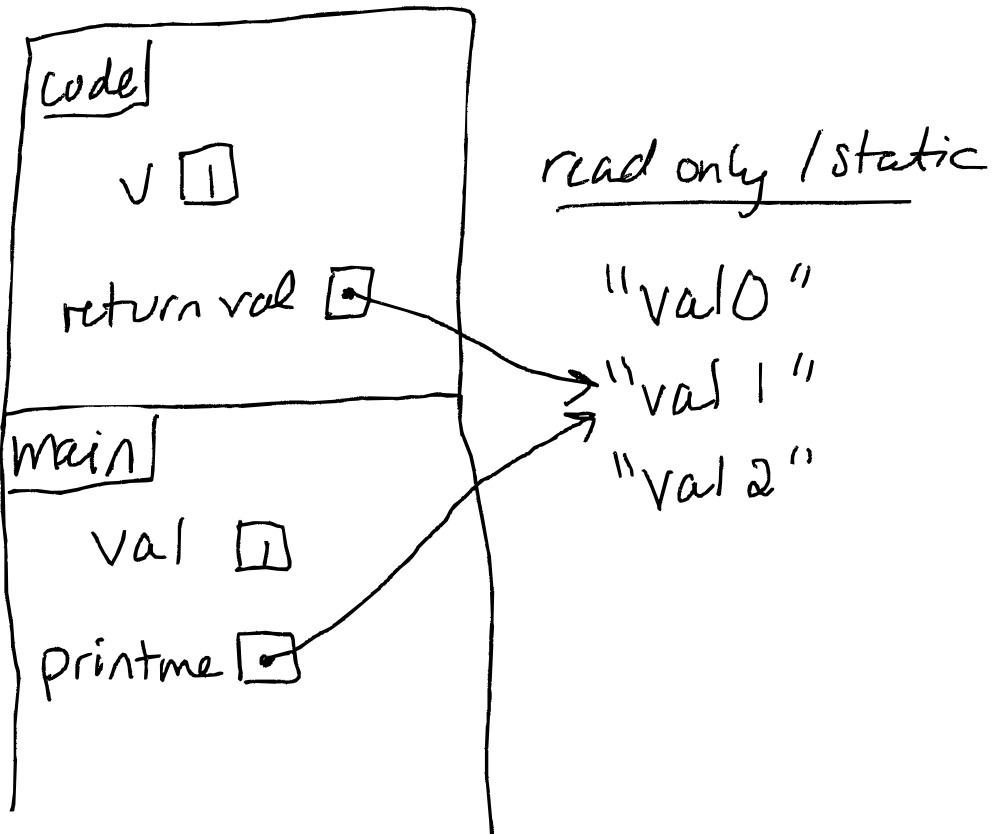
"val0"

"val1"

"val2"

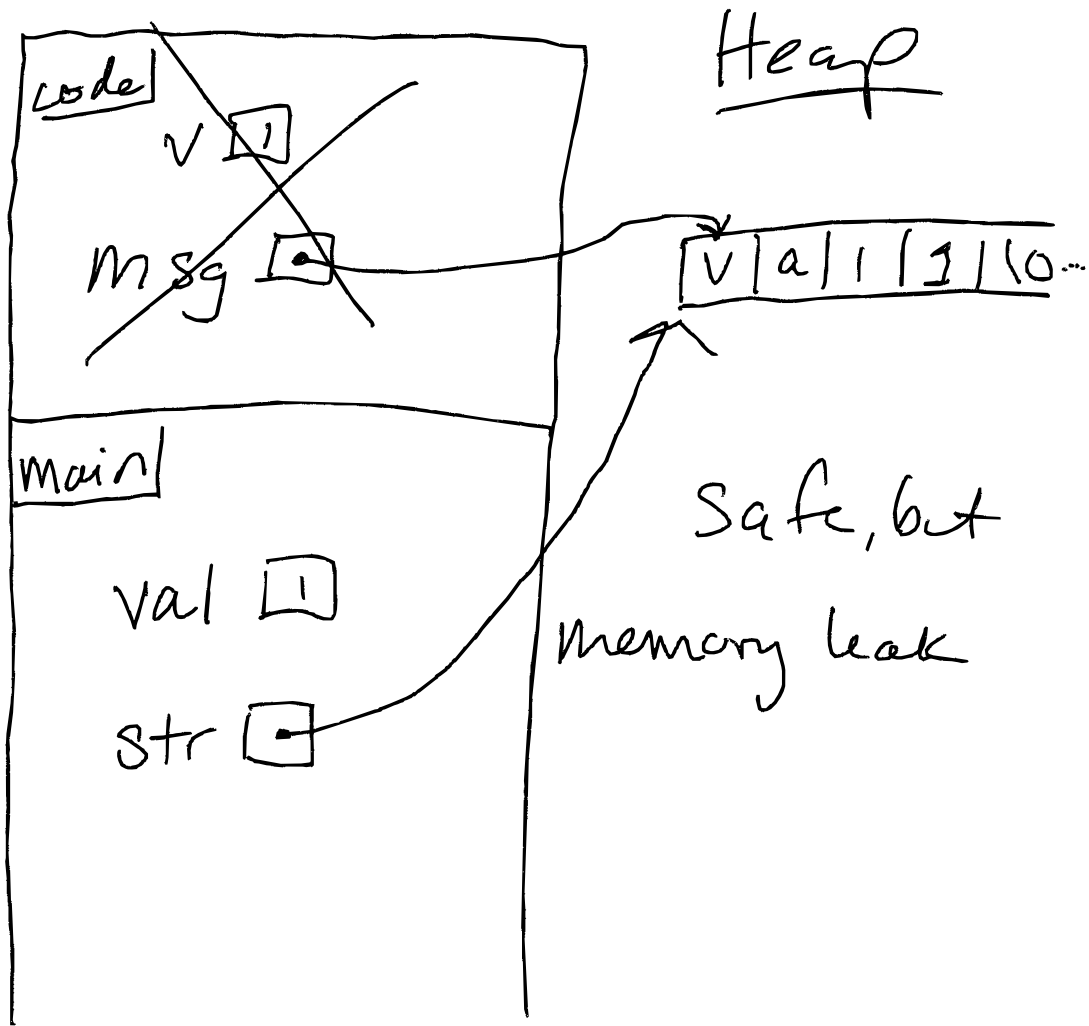
Return value gotchas #2

Safe ✓



Return value gotchas

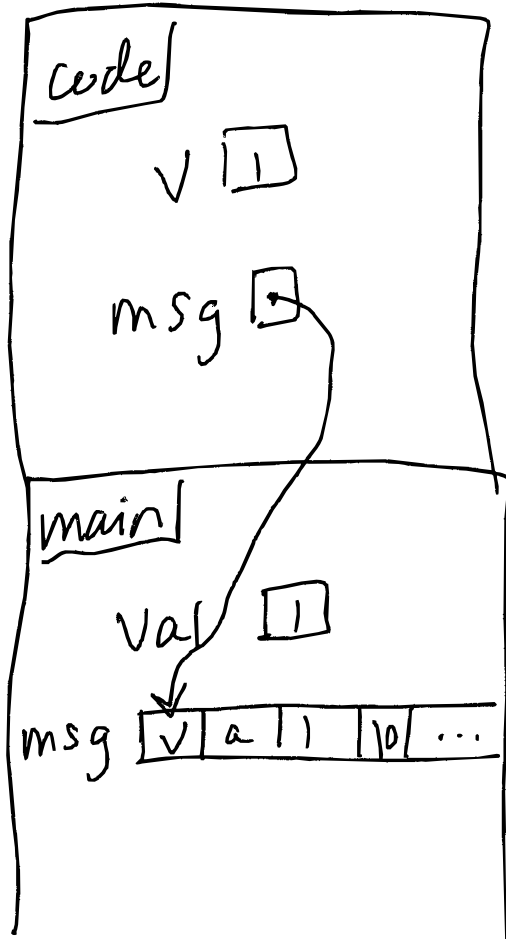
3



Return value gotchas

#4

safe ✓



passing a
ptr (b/c
array)

~~Agenda~~

~~Announcements~~

~~* questions from other lectures~~

~~Review material~~

- code for new material (rand)
- review new material
- double check these examples
- double check slides
- copy C examples to remarkable

